

Modified Palmer–Mansoori Permeability Model

Python Implementation of an Academic Engineering Model

Domain: Petroleum Engineering — Coalbed Methane (CBM)

Focus: Permeability Modeling & Parameter Sensitivity

Tools: Python (NumPy, Pandas, Matplotlib)

Original Context: B.Tech Major Project — Excel & MATLAB

Author: Agamya Jain

Project Background: The original academic project was implemented using Excel and MATLAB as part of my B.Tech major project. For my portfolio, I recreated the model calculation workflow in Python without changing the underlying engineering formulation.

1. Project Overview

In coalbed methane (CBM) reservoirs, permeability changes as reservoir pressure changes during depletion. Two important mechanisms influencing this behavior are cleat compression and coal-matrix shrinkage.

This project uses the Modified Palmer–Mansoori permeability model to study how these effects influence porosity and permeability under different combinations of initial porosity and Young's modulus.

For the portfolio version, the original engineering formulation was recreated in Python. The implementation automates scenario calculations, organizes the results into structured datasets, and generates comparative visualizations.

Objectives

- Recreate the academic model calculation in Python.
- Analyze the effect of Young's modulus and initial porosity on permeability response.
- Evaluate four parameter scenarios across a defined pressure range.
- Generate structured CSV outputs and visualizations.
- Verify the Python results against the original Excel model.

2. Model Formulation

The Modified Palmer–Mansoori model represents the change in porosity resulting from pressure-dependent deformation and matrix-shrinkage effects.

$$\frac{\phi}{\phi_i} = 1 + \frac{(1+\nu)(1-2\nu)}{(1-\nu)E\phi_i}(p - p_i) + \frac{c_0}{\phi_i} \frac{2(1-2\nu)}{3(1-\nu)} \left(\frac{p_i}{p_i + p_L} - \frac{p}{p + p_L} \right)$$

$$\frac{k}{k_0} = \left(\frac{\phi}{\phi_0} \right)^3$$

The formulation combines the pressure-dependent deformation response with the matrix-shrinkage contribution, and the resulting porosity ratio is converted to permeability using the Carman–Kozeny relationship.

Model Parameters

Parameter	Symbol	Description
Current pressure	p	Pressure at which the model is evaluated
Initial pressure	p_i	Reference reservoir pressure
Langmuir pressure	p_L	Langmuir pressure parameter
Young's modulus	E	Elastic property of the coal matrix
Poisson's ratio	ν	Poisson's ratio
Initial porosity	ϕ_i	Reference porosity
Initial compressibility	c_0	Initial compressibility parameter
Dynamic porosity	ϕ	Porosity calculated by the model

Permeability ratio	k/k_i	Dynamic permeability relative to initial permeability
--------------------	---------	---

3. Inputs & Scenario Design

The Python implementation uses the same parameter combinations as the original academic model.

Common Parameters

Parameter	Value	Unit
Initial pressure (p_i)	1100	psia
Langmuir pressure (p_L)	625	psia
Initial compressibility (c_0)	0.013	psia ⁻¹
Poisson's ratio (ν)	0.39	—

Scenario Matrix

Scenario	Initial Porosity ϕ_i	Young's Modulus E
S1	0.001	124,000 psi
S2	0.001	445,000 psi
S3	0.005	124,000 psi
S4	0.005	445,000 psi

Pressure Range: 0 to 2,000 psia in 100-psia increments, giving 21 pressure points for each scenario.

4. Python Implementation

The engineering formulation was recreated as a reusable Python function and applied consistently across all four scenarios.

```
def palmer_mansoori_model(p, pi, pL, E, nu, phi_i, c0):
    phi_ratio = (
        1
        + ((1 + nu) * (1 - 2 * nu))
        / ((1 - nu) * E * phi_i) * (p - pi)
        + (c0 / phi_i)
        * (2 * (1 - 2 * nu) / (3 * (1 - nu)))
        * ((pi / (pi + pL)) - (p / (p + pL)))
    )
    k_ratio = phi_ratio ** 3
    return phi_ratio, k_ratio
```

Computational Workflow

- Define model parameters and pressure range.
- Create the four parameter scenarios.
- Calculate ϕ/ϕ_i and k/k_i at every pressure point.
- Store the results in Pandas DataFrames.
- Export structured CSV outputs.
- Generate pressure-response plots using Matplotlib.

The generated dataset contains Pressure_psia, E, phi_i, phi_ratio, and k_ratio, making the outputs easy to inspect and compare.

5. Results & Key Findings

The four scenarios show that both Young's modulus and initial porosity influence the calculated permeability response.

Scenario	ϕ_i	E (psi)	Key Result
S1	0.001	124,000	Strong response; negative ratios occur under extreme depletion
S2	0.001	445,000	$k/k_i \approx 0.708$ at 700 psia; ≈ 5.396 at 0 psia
S3	0.005	124,000	$k/k_i \approx 0.128$ at 100 psia; slight rebound at 0 psia
S4	0.005	445,000	Minimum $k/k_i \approx 0.936$ at 700 psia; ≈ 1.524 at 0 psia

Key Findings

- **Young's modulus:** Higher Young's modulus produces a smaller permeability decline during depletion.
- **Initial porosity:** Lower initial porosity increases the sensitivity of the calculated response because ϕ_i appears in the denominator of the pressure-dependent terms.
- **Permeability recovery:** Some scenarios show an initial decline followed by recovery at lower pressures as the balance between the modeled deformation mechanisms changes.
- **Nonphysical output:** Scenario 1 produces negative porosity and permeability ratios under extreme depletion. These values were retained rather than clipped so that the original analytical formulation is reproduced faithfully.

Illustrative Scenario — S4: At 1100 psia, $k/k_i = 1.000$. It falls to approximately 0.936 at 700 psia and then rises to approximately 1.524 at 0 psia, demonstrating a decline followed by permeability recovery.

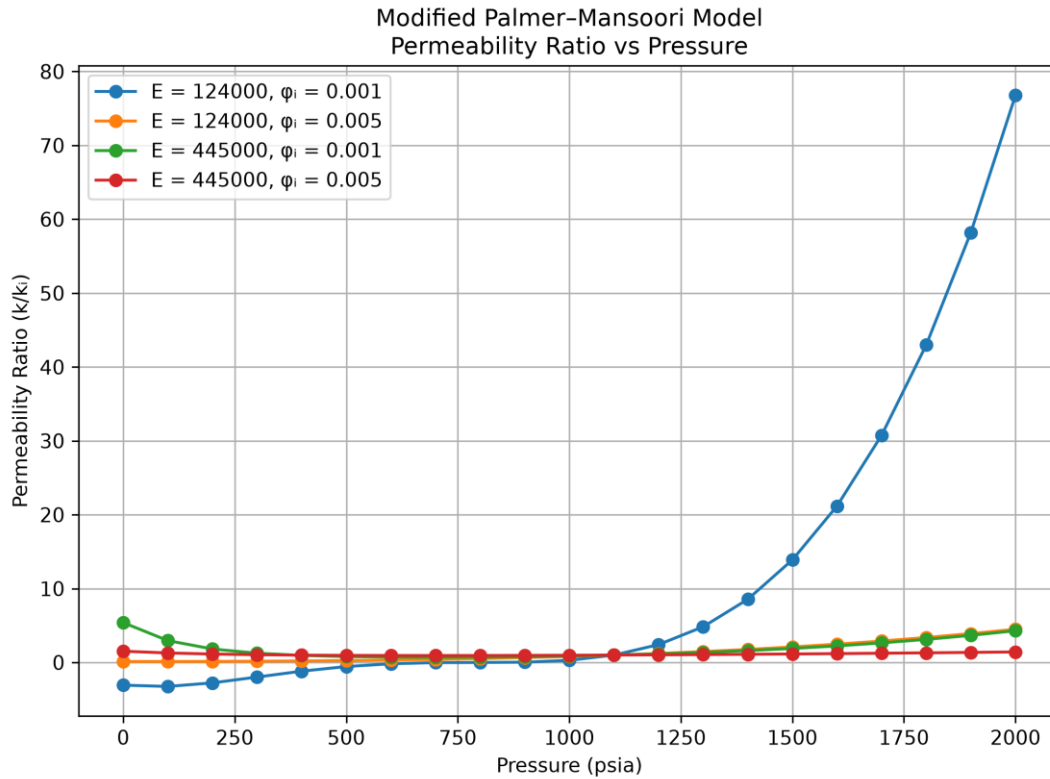


Figure 1: Modeled permeability ratio versus pressure for the four parameter scenarios.

6. Validation & Limitations

Validation

- The Python implementation was verified against the original Palmer–Mansoori Excel model across all four scenarios and 21 pressure points per scenario.
- The calculated porosity and permeability ratios closely reproduce the original Excel outputs.
- Small numerical differences arise because Python uses the exact $2/3$ factor, while the original implementation uses the rounded value 0.6667.
- These differences do not change the observed pressure-response trends or scenario-level conclusions.

Limitations

- The analysis uses an analytical model rather than a history-matched field production case.
- Results depend on the selected reservoir and rock-property parameters.
- Young's modulus and Poisson's ratio are treated as constant across the pressure range.
- The model does not include two-phase gas-water relative permeability effects.
- The unconstrained formulation can produce nonphysical negative ratios under some extreme parameter combinations.
- The results are intended for model interpretation and parameter sensitivity analysis rather than direct field-performance prediction.

7. Conclusion

This project recreates an academic Modified Palmer–Mansoori CBM permeability model as a Python-based computational workflow.

The original model was implemented using Excel and MATLAB for the B.Tech major project. For the portfolio adaptation, the same engineering formulation was recreated in Python and evaluated across four combinations of initial porosity and Young's modulus. The analysis shows that Young's modulus and initial porosity strongly influence the calculated permeability response. Higher Young's modulus generally produces a smaller permeability decline, while lower initial porosity increases model sensitivity.

The Python implementation provides a reproducible way to run multiple scenarios, generate structured outputs, and visualize the resulting pressure-response behavior.

8. Technology Used & Portfolio Note

Technology	Purpose
Python	Core language for the computational workflow
NumPy	Numerical calculations
Pandas	Scenario management and result processing
Matplotlib	Pressure-response visualization

Portfolio Note: This repository is a portfolio adaptation of an academic engineering project. The underlying engineering formulation and scenario design are based on the original B.Tech project. The Python implementation was created separately to demonstrate computational modeling, numerical analysis, structured data handling, scenario analysis, and visualization.